



Московский Государственный Университет им. М. В. Ломоносова

Факультет Вычислительной Математики и Кибернетики.

Кафедра информационной безопасности.

Александров Богдан Георгиевич

**«Исследование методов автоматической генерации
входных данных для тестирования
модулей обработки шаблонов веб-страниц»**

Выпускная квалификационная работа

Научный руководитель:

младший научный сотрудник

Петухов Андрей Александрович

Москва, 2022

Содержание

1	Введение	4
1.1	Вступление	4
1.2	Актуальность	4
1.3	Цель работы	5
1.4	Постановка задачи	5
1.5	Используемая терминология	5
2	Анализ задачи	7
2.1	Принципы работы шаблонизатора	7
2.2	Безопасность шаблонизаторов	9
2.3	Функциональность связанная с безопасностью	10
2.4	Исследуемая функциональность	11
2.5	Требования к фаззингу шаблонизаторов	11
3	Фаззинг	12
3.1	Методы фаззинга на основе информации целевой программы	12
3.2	Методы генетического программирования	13
3.3	Способы генерации входных данных	13
3.4	Критерии сравнения инструментов для фаззинга	14
4	Обзор статей про фаззинг	16
4.1	Рассмотренные статьи	16
4.2	Итоги	20
5	Экспериментальные исследования	22
5.1	Покрытие кода	22
5.2	Найденные недостатки	26
5.3	Интерпретация результатов	27
5.4	Итоги эксперимента	27
5.5	Вывод	29

1. Введение

1.1. Вступление

С развитием интернета множество компаний и организаций стало использовать его для предоставления различных сфер услуг, информации и новостей. Как правило, в этих целях создаются веб-приложения, посредством которых и предоставляется некоторый контент пользователям. Для быстрого и удобного создания этих приложений были созданы различные инструменты и фреймворки, частью которых зачастую являются модули обработки шаблонов веб-страниц. Такие инструменты называются шаблонизаторами. Они предоставляют разработчикам удобный интерфейс для написания веб-приложений, а также встроенные методы и функции для обработки данных.

Важным аспектом веб-сервисов является безопасность, и в современных фреймворках зачастую уже реализованы инструменты для защиты от веб-атак. Задача программиста - правильно их использовать, так как некоторые уязвимости приложения появляются по вине самого веб-разработчика. Но даже при правильном использовании всех инструментов в программном продукте могут быть выявлены недостатки в защите, это может произойти уже из-за ошибок при разработке фреймворка. В данной работе рассматриваются именно уязвимости, связанные с самими шаблонизаторами, а также применяется фазз-тестирование для их обнаружения.

1.2. Актуальность

Шаблонизаторы являются очень удобными инструментами для того, чтобы быстро написать свое веб-приложение, вследствие чего они пользуются популярностью. Все больше веб-приложений пишется на их основе, поэтому безопасность этих фреймворков является актуальной на сегодняшний день. Для тестирования метод фаззинга был выбран не случайно - его применение показало эффективность при нахождении различных уязвимостей в движках, компиляторах и других обработчиках кода, как показано в статьях 1 и 2.

1.3. Цель работы

Целью работы является изучение применимости методов fuzz-тестирования к шаблонизаторам. Для этого необходимо рассмотреть предметную область фазз-тестирования, изучить статьи про фаззинг, выбрать методы, подходящие для шаблонизаторов и исследовать на практике их применимость.

1.4. Постановка задачи

- Исследование методов фазз-тестирования
 - Изучить предметную область фаззинга
 - Изучить статьи о фазз-тестировании, рассмотреть его вариации и техники
 - Составить требования к будущему инструменту для фаззинга на основе рассмотренных статей
- Поиск готовых инструментов
 - Выбрать фреймворк для языка PHP, который в дальнейшем будет подвержен фаззингу
 - Выяснить, какие инструменты для фаззинга существуют на сегодняшний день
 - Отобрать из готовых фазз-инструментов подходящие под критерии, основанные на разборе статей
- Провести экспериментальные исследования на выбранном шаблонизаторе, сделать выводы о применимости фаззинга для обработчиков шаблонов.

1.5. Используемая терминология

Шаблонизатор— программное обеспечение, позволяющее использовать html-шаблоны для генерации конечных html-страниц. Основная цель использования шаблонизаторов — это отделение представления данных от исполняемого кода. Это необходимо для обеспечения возможности параллельной работы программиста и дизайнера-верстальщика, что ускоряет процесс разработки продукта. Использование шаблонизаторов улучшает читаемость кода и внесение изменений во внешний вид, когда проект целиком выполняет один

человек.

Веб-приложение — клиент-серверное приложение, в котором клиент взаимодействует с веб-сервером при помощи браузера. Логика веб-приложения распределена между сервером и клиентом, хранение данных осуществляется, преимущественно, на сервере, обмен информацией происходит по сети.

Фазз-тестирование — техника тестирования программного обеспечения, заключающаяся в передаче приложению на вход неправильных, неожиданных или случайных данных, генерирующихся автоматически. Предметом интереса являются падения и зависания, нарушения внутренней логики и проверок в коде приложения, утечки памяти, вызванные такими данными на входе.

Уязвимость - свойство приложения, позволяющее одному его пользователю (атакующему) повредить безопасности данных других его пользователей. Под вредом безопасности понимается нарушение одного из трех принципов безопасности информации: конфиденциальности, целостности или доступности

Фреймворк - программная платформа, определяющая структуру программной системы; программное обеспечение, облегчающее разработку и объединение разных компонентов большого программного проекта.

2. Анализ задачи

2.1. Принципы работы шаблонизатора

Шаблонизатор – инструмент для создания веб-приложений, который генерирует html-страницы на основе шаблонов. Шаблоном является файл, содержащий html-разметку и конструкции обработчика шаблонов - специальные поля, либо же теги. Также ими могут быть различные выражения для прохода по массиву данных, вычисления несложных арифметических операций, проверка условия, все это зависит от функциональности шаблонизатора.

Процесс обработки шаблона состоит из следующих этапов:

- Лексический анализ

На этом этапе происходит разбор входной последовательности символов на rozpoznанные группы — лексемы — с целью получения на выходе идентифицированных последовательностей, называемых «токенами». Примером токенов могут послужить переменные шаблонизатора или разметка html, которая не претерпевает изменений в ходе динамической подстановки данных.

- Синтаксический анализ

После лексического анализа шаблонизатор сопоставляет полученную последовательность лексем с его формальной грамматикой. В ходе данного этапа текст переводится в структуру под названием абстрактное синтаксическое дерево

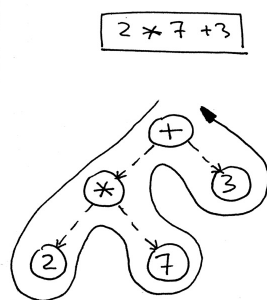


Рис. 1: Абстрактное синтаксическое дерево

- Семантический анализ

Проверка на корректность использования конструкций шаблонизатора, например инициализация переменных перед их использованием, соотношение операций и типов объектов.

- Создание конечного кода

По итогу работы анализа входного шаблона, на его основе генерируется код, который выдает посетителю сайта html-страницу.

Рассмотрим функционал Smarty, шаблонизатор для языка PHP:

```
<?php
\\создание объекта, который реализует все функции шаблонизатора
$smarty = new Smarty();
\\ создаем переменную firstname со значением Doug
$smarty->assign('firstname', 'Doug');
\\ создаем переменную lastname со значением Evans
$smarty->assign('lastname', 'Evans');
\\ создаем переменную meetingPlace со значением New York
$smarty->assign('meetingPlace', 'New York');
\\ вызываем функцию, которая генерирует html-страницу, и подаем ей на
вход шаблон.
$smarty->display('index.tpl');
?>
```

Рис. 2: Код Smarty

Шаблон, который используется для генерации html-страницы:

```
<br>
Hello {\$firstname} {\$lastname}, glad to see you can make it.
<br/>
This weeks meeting is in {\$meetingPlace}
```

Рис. 3: html Шаблон

Шаблон Freemarker (шаблонизатор для языка Java) похож на тот же, что у Smarty:

```
<!DOCTYPE html>
<html lang="en»
<head> <meta charset="UTF-8» <title>$name!</title> </head>
<body> <h2>Hello $name!</h2> </body>
</html>
```

Код, ответственный за обработку шаблона:

```
import freemarker.template.Configuration;
import freemarker.template.Template;
...
// Конфигурация
Configuration cfg = new Configuration(Configuration.VERSION_2_3_27);
// модель данных
Map<String, Object> root = new HashMap<>();
root.put("name", "Freemarker");
// шаблон Template temp = cfg.getTemplate("test.ftl");
// обработка шаблона и модели данных
Writer out = new OutputStreamWriter(System.out);
// вывод в консоль temp.process(root, out);
```

Рис. 4: html Шаблон

2.2. Безопасность шаблонизаторов

Шаблонизаторы предоставляют некоторые инструменты для безопасной работы с данными. Обрабатывать данные, полученные из Интернета, очень важно, так как злоумышленник потенциально может выполнить вредоносный код на сервере, либо провести атаку XSS.

Для защиты от выполнения вредоносного кода в шаблонизаторе Twig используется такой инструмент как песочница. Песочница — специально выделенная (изолированная) среда для безопасного исполнения компьютерных программ. Обычно представляет собой жёстко контролируемый набор ресурсов. Другие шаблонизаторы ограничивают множество методов, которые можно явно вызывать в шаблоне.

Для защиты от XSS используется экранирование символов. Данные, полученные из Интернета, кодируются особым образом, в соответствии с html, url или JavaScript сценарием. Относительно шаблонизаторов также применима атака SSTI, или внедрение шаблона. Она реализуема, когда разработчик веб-приложения неправильно использует API шаблонизатора, и в качестве шаблона использует введенные пользователем данные. Данный код как

раз иллюстрирует такой случай:

$$\$output = \$twig \rightarrow$$

```
render($_GET['custom_email'], array("first_name" => $user.first_name));
```

Метод `render` генерирует шаблон, состоящий из одного email адреса, никаких других тегов, html-разметок в шаблоне нет. Если передать в GET-параметр «`custom_email`» значение `{7*7}`, то пользователь увидит результат умножения – число 49, таким образом на сервере выполнялась арифметическая операция и результат использовался при формировании html-страницы. В рамках данной работы нас не будут интересовать ошибки разработчиков, интерес представляют лишь потенциальные уязвимости и некорректность в логике самого шаблонизатора. Поэтому рассматриваться будут только те методы шаблонизаторов, которые ответственны за безопасность веб-приложения.

2.3. Функциональность связанная с безопасностью

1. RCE

Для предотвращения выполнения зловредного кода на сервере, шаблонизаторы либо запрещают явно вызывать некоторые методы в шаблоне, либо предоставляют такой инструмент как песочница. Песочница может работать автоматически, то есть не требовать от разработчика никаких действий, либо ее необходимо настроить вручную. Пример подключения песочницы шаблонизатора Twig:

```
{%sandbox%}
```

конструкции шаблонизатора

```
{%sandbox%}
```

2. XSS

Для предотвращения данной атаки шаблонизаторы используют экранирование символов, то есть их кодировку по определенному сценарию, чтобы данные, введенные пользователем, не обрабатывались браузером, ввиду их потенциальной опасности. Экранирование может быть автоматическим либо требует явного указания, также оно может выполняться по разным сценариям, в зависимости от того, обработчик какого языка мы хотим обезопасить – html, url, JavaScript. В шаблонизаторе

Freemarker экранирование устанавливается следующим образом:

```
<#ftl output_format = "XML">
```

Все конструкции шаблонизатора после данной строки будут экранировать данные по сценарию XML. Большинство же рассмотренных шаблонизаторов автоматически экранирует данные в соответствии с html-кодировкой.

2.4. Исследуемая функциональность

В большинстве своем фреймворки запрещают использование конструкций языка в шаблонах и допускают только свои собственные, например в 6 и 7. Более того, при подстановке данных, они воспринимаются просто как текст и не рассматриваются как исполняемый код, так что угроза RCE по итогу просто невозможна.

В то же время угроза XSS атаки присутствует всегда, так как не зависит от конструкций языков программирования и фреймворков. Поэтому шаблонизаторы всегда должны обрабатывать данные и корректно отображать их. Таким образом, в данной работе внимание уделяется именно той функциональности фреймворков, которые ответственны за экранирование данных. Интерес представляет как автоматическая защита, так и подключаемые разработчиком функции. Тестирование ее на наличие уязвимостей адресуется фазз-тестированию.

2.5. Требования к фаззингу шаблонизаторов

Тесты, которые будут подаваться шаблонизатору на вход, будут подвергнуты лексическому, синтаксическому и семантическому анализу, описанным выше. Чтобы они смогли задействовать функционал, ответственный за экранирование, необходимо их соответствие синтаксису языка шаблонов. В противном случае программа будет выдавать синтаксические ошибки, а более глубокий код не будет затронут. Таким образом инструментам фазз-тестирования предстоит задача - генерировать достаточно валидные данные, чтобы пройти первые этапы анализа, но достаточно сложные для возможности спровоцировать некорректное поведение шаблонизатора. Также в тестах должны содержаться различные комбинации конструкций, использующие инструменты экранирования.

3. Фаззинг

Общая схема процесса фаззинга представлена на рисунке. Начальный корпус тестов либо собран руками, либо сгенерирован с помощью грамматики. Затем эти данные подаются на вход фазз-инструменту, который проводит некоторые преобразования над ними и подает целевой программе. В ходе тестирования собирается информация о поведении программы, ее ошибках и падениях, затем на основе полученной информации интересные экземпляры отбираются, используются для мутации, далее создается новое поколение, и все это проводится в несколько итераций.

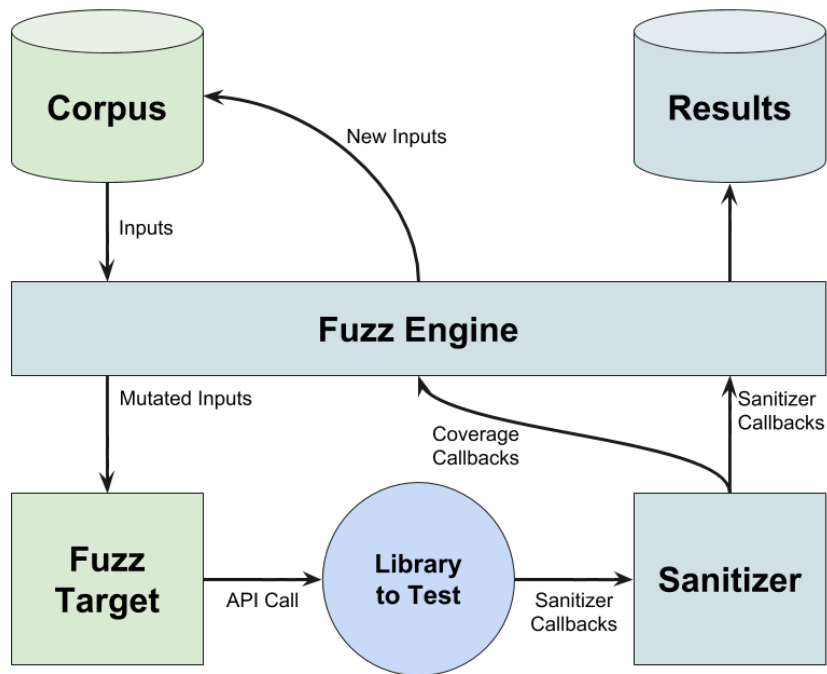


Рис. 5: Общая схема фаззинга

3.1. Методы фаззинга на основе информации целевой программы

По информации о целевой программе, фазз-тестирование можно разделить на три вида: метод белого, черного или серого ящика. Метод белого ящика подразумевает под собой владение полной информацией о тестируемой программе, ее исходным кодом, детали реализации функций и используемых инструментах. При методе черного ящика наоборот,

никакой исходной информации нет, все, что можно получить от тестируемой программы, это ее результат действия на полученные входные данные. Метод серого ящика является чем-то средним, между черным и белым, то есть владение частью информации. Например, информированность о структуре входных данных, или о некоторых инструментах, которые использует целевая программа, а детали реализации уже неизвестны

3.2. Методы генетического программирования

При генерации тестов, фаззеры используют такие методы, как генетическое программирование и мутацию. Генетическое программирование подразумевает под собой генерацию данных, при которой используется информация, полученная от тестируемой программы во время выполнения на предыдущем множестве тестов. Таким образом, новое поколение входных данных призвано улучшить результаты тестирования, по сравнению с предыдущими, например, покрыть большее количество кода, вызвать больше исключений и падений.

Мутация тестов – такая техника, при которой в исходных тестах происходят случайным образом некоторые изменения: замена подстроки, некоторых байтов, конструкции языка программирования. Для мутации, как правило, используют не все множество входных данных, а только часть, отобранных на основе некоторых критериев, например, по значению на fitness-функции. Fitness-функция, или функция приспособленности, для каждого теста выдает вещественное или целочисленное значение, которое отображает, насколько он интересен в плане потенциала вызвать исключение или ошибку, насколько хорош для генерации следующей популяции тестов. Отобранные тесты называются «семенами», на их основе генерируется новое поколение входных данных: на каждом выбранном экземпляре производят ряд мутаций, полученные данные подаются на вход целевой программе. Таким образом, мутационный процесс происходит итерационно. Правила для мутации являются детерминированными, и задаются до начала fuzz-тестирования.

3.3. Способы генерации входных данных

При фазз-тестировании необходимо определить структуру тестовых случаев, которые будут подавать на вход целевой программе. Самое простое решение – подавать случайную последовательность байтов. Но это неэффективно в случаях, когда тестируемая програм-

ма принимает на вход данные с определенной структурой, как показано в 1, так как такие объекты просто не пройдут этапы лексического и синтаксического анализатора и не смогут задействовать функции и методы, реализующие логику работы самой программы. В таких случаях извлекается грамматика валидных входных данных, и на основе ее уже строятся тесты.

Чтобы эффективно использовать фазз-тестирование и направить его на генерацию хороших тестовых случаев, необходимо производить сбор информации, полученной от выполнений целевой программы на входных данных. Для этого анализируются покрытие кода, ошибки и падения, далее их фильтрация, отсеивание тех, которые получены на этапе проверки соответствия синтаксису. На основе собранной информации генерируются новые поколения тестов.

3.4. Критерии сравнения инструментов для фаззинга

1. Какие программы являются целевыми?

Важно, чтобы целью фаззинга была программа, работающая схожим образом с шаблонизатором, и которая принимает данные с четкой структурой. Например, движки JavaScript или компиляторы.

2. Откуда взять примеры входных данных? Какие данные взять в качестве первого поколения в итерационном процессе?

Чтобы начать процесс фазз-тестирования, нужно задать начальный набор тестов. Первое поколение в фаззинге довольно важно, так как взяв изначально подходящие входные данные, можно быстрее обнаружить ошибки, и вектор генерации будет направлен в сторону перспективных с точки зрения обнаружения недостатков экземпляров.

3. Как происходит мутация?

Процесс мутации должен быть построен таким образом, чтобы каждое следующее поколение тестов было лучше предыдущего, вызывало новые ошибки, покрывало новое пространство кода. Важным качеством мутационного процесса является способность производить входные данные с подходящей структурой, в противном случае эти тесты будут бесполезны с точки зрения проверки корректности логики работы

шаблонизатора.

4. Как отбирать семена для следующей итерации мутации?

Под семенами подразумевается выборка тестов, на основе которых строится следующее поколение. Правильный выбор семян направит фаззинг в правильном направлении, и вероятность найти ошибку в коде возрастет.

5. Какую fitness-функцию использовать?

Для метрики качества тестовых случаев используется функция приспособленности. Рассматривались функции, которые были предложены в статьях. Особое внимание уделялось лишь тем, которые применимы в отношении шаблонизаторов.

4. Обзор статей про фаззинг

Для изучения методов и подходов фазз-тестирования, был рассмотрен ряд статей на эту тему. В ходе обзора были сформированы критерии, по которым были отобраны статьи с наиболее интересными и потенциально применимыми к шаблонизаторам техниками тестирования:

4.1. Рассмотренные статьи

1. «A Review on Grammar-Based Fuzzing Techniques»

В данной статье не акцентируется внимание на каком-либо фаззере, а сравниваются они по целевой программе – обработчики XML текстов и JavaScript кода. В основе своей, рассматривается Grammar-based техника. Ее суть в том, чтобы извлечь грамматику тестируемой программы, и на ее основе генерировать данные для программы. Преимущество техники в том, что ей нужно относительно мало времени и ресурсов для того, чтобы создать большое количество тестов. Минус - зависимость от примеров входных данных для тестируемой программы. Этой технике уделяется внимание потому, что она эффективна для создания множества тестовых наборов автоматически, как показано в 1, 2 и 4.

Грамматика может извлекаться как с помощью существующих инструментов, например, Grammarinator, Gold Parser, так и собственных программ.

В процессе тестирования используют мутацию. В качестве первого поколения используются данные, которые ввели пользователи Интернета, либо сгенерированные на основе грамматики. Они подаются на вход фаззеру, который с помощью парсера разбивает эти примеры на токены и строит абстрактные синтаксические деревья (AST) для каждого теста, в котором узлами и листьями являются некоторые сущности: объекты и связывающие их операции и конструкции. После завершения работы целевой программы на первой итерации тестов, в построенных AST случайным образом выбираются некоторые узлы, и заменяются на другие, но того же грамматического типа. Таким образом синтаксис входных данных не нарушается, что позволяет им сохранять правильную структуру.

В качестве будущей работы, авторы хотят создать инструмент, который будет ана-

лизовать пользовательский вход в программу. Пользовательский ввод не всегда валиден, и получив грамматику из пользовательского входа, можно создавать интересные тесты, которые, возможно, вызовут некоторые ошибки в программе.

2. «Evolutionary Grammar-Based Fuzzing»

Целью этой статьи является изучение обработчиков данных JSON, а также движков JavaScript и CSS. Для их тестирования был построен фаззер под названием EvoGFuzz.

Данный инструмент зависит от грамматики входов целевой программы и от изначального корпуса. На его основе он вычисляет вероятностное распределение правил, которое позволяет строить тесты, похожие по структуре на те примеры, на которых обучался фаззер. Также данный инструмент использует coverage-guided технику, то есть его цель – покрыть наибольшее количество кода. Мотивация в том, что новые пути выполнения программы могут показать нам неизведанные ранее ошибки.

Перед процессом тестирования грамматика извлекается с помощью алгоритмов машинного обучения. Они помогают в изучении структуры входных данных, и на основе этого нейронная сеть может выдавать на тестовые примеры.

В процессе фаззинга, как и в предыдущей статье, используется мутация семян. Для отбора лучших экземпляров используются эволюционные алгоритмы. Fitness-функция рассчитывается по следующей формуле:

$$fitness(x) = score_feedback(x) + score_structure(x)$$

Первое слагаемое - функция обратной связи, являющейся дискретной. Она равна либо нулю, если тест не вызвал никаких исключений в ходе выполнения программы, либо бесконечности, если исключение вызвалось. То есть те экземпляры, на которых были падения, фаззер однозначно считает интересными и будет всегда включать в семена для генерации следующего поколения тестов.

$$score_feedback(x) = \begin{cases} inf & \text{if } x \text{ triggered exception} \\ 0 & \text{otherwise} \end{cases}$$

Функция структуры вычисляется как квадрат количества разных расширений(грамматик), использованных при построении тестового файла, деленное на его

длину, помноженное на лямбду - некоторый коэффициент, который можно регулировать.

$$score_structure(x) = ratio(x) * expansions(x)$$

$$ratio(x) = \frac{expansions(x)}{\lambda * length(x)}$$

Тестовые наборы сортируются в порядке убывания по значению fitness-функции, затем отбирается некоторый процент лучших из них. Далее к ним добавляется некоторый набор из уже выигравших ранее данную гонку наборов. По итогу, имеется корпус тестовых входов, к которым примешивается несколько случайных верных входов для тестируемой программы. Это делается с целью предостеречь мутационный процесс от генерации тестов с неверной входной структурой. Затем вычисляется вероятностная грамматика на основе этих входных данных, то есть рассчитывается распределение правил, используемых в тестовых случаях. По этой вероятностной грамматике строится новое поколение входных данных.

По итогам статьи, построенный фаззер показал следующие результаты: авторы показывают, что EvoGFuzz улучшает покрытие кода для всех субъектов и способен увеличить медианное покрытие линии для JSON до 18,43%, для JavaScript до 47,93% и для CSS3 до 8,45%. Субъектами являются обработчики JavaScript, CSS, JSON: Argo, Genson, Gson, JSONJav, JsonToJava, minimalJSON, Pojo, Rhino, cssValidator.

3. «Efficient approach to fuzzing interpreters»

В данной статье рассматривается такой фаззер, как Fluff, ориентированный на тестирование движков JavaScript.

В качестве метрики "хорошести" теста здесь также используется покрытие кода. Чем больше новых путей обхватывает тестов, тем лучше. Также Fluff использует грамматику входной структуры для генерации тестов. Для ее извлечения используется Grammarinator.

Fluff хранит у себя абстрактное синтаксическое дерево, которое он построил от сериализованного AST, полученное от парсера тестируемого интерпретатора. Построение дерева происходит до начала тестирования.

В статье предлагаются несколько видов мутации семян:

- изменение случайной последовательности байтов

- изменение случайной последовательности ASCII-символов
- изменение некоторых выражений в программе JavaScript.
- мутация дерева AST по аналогии, описанной в первой статье. Иногда на вход программе предлагается подавать данные с "испорченными" байтами с той идеей, что это поможет вызвать неожиданные сбои.

Предпочтение отдается последним двум способам, так как первые два редко проходят стадию анализа структуры. Так как в качестве метрики используется покрытие кода, последующие поколения тестов генерируются на основе тех, что имеют наибольшие покрытия или новые участки. Для сбора информации о выполнении работы программы на входном экземпляре используется open-source инструмент AFL, который также является фаззером, в котором реализован инструмент мониторинга целевой программы.

В качестве будущей работы, авторы планируют расширить возможность тестирования фаззера Fluff на компиляторы, увеличить набор тестируемых языков. Также они хотят встроить функцию, которая бы проверяла стандарты языка, проверяя правильно или нет работает тестируемая программа.

4. «Learn&Fuzz: Machine Learning for Input Fuzzing»

В данной статье описывается подход, схожий по работе с Grammar-based фаззерами. Также используется статистическое распределение грамматики для генерации тестов. Целевыми программами являются PDF viewers, для тестирования которых был построен фаззер Learn&Fuzz.

Для генерации тестов в этой статье предлагается использовать нейронную сеть. Используется seq2seq модель – это модель, принимающая на вход последовательность элементов (слов, букв, признаков изображения) и возвращающая другую последовательность элементов. Таким образом, чтобы обучить сеть, использовались несколько кусков PDF файлов, их объединяли в один, затем разбивали на последовательность символов, деля на куски равного размера, и подавали на вход нейронной сети.

Способов генерации тестов несколько:

- (a) Можно несколько раз сгенерировать следующий элемент последовательности, и выбрать из полученных наиболее встречающийся. Такой способ будет генери-

ровать хорошие PDF файлы, но это не позволит результативно провести фазз-тестирование, так как большинство тестов будут абсолютно верными с точки зрения синтаксиса и семантики.

- (b) Генерация нескольких следующих элементов последовательности. Данный способ может генерировать невалидные PDF файлы, но это позволяет проверить, хорошо ли обрабатываются файлы такого формата, вызывает ли PDF viewer ошибки на неверных данных.
- (c) Метод, совмещающий два предыдущих. Выбирается лучший элемент для продолжения последовательности, а когда генерируется символ конца строки, для начала новой последовательности создается уже случайный элемент. Затем последовательность снова пополняется наивероятнейшими элементами. Но если генерировать постоянно валидные PDF файлы, то покрыть максимальное количество кода будет затруднительно - не будет покрыта часть кода, ответственная за обработку неверных файлов. Для устранения данного недостатка предлагается немного портить некоторые экземпляры. Например, следующим элементом последовательности использовать не лучший, а худший элемент.

Также, для того, чтобы данные были не всегда верны по структуре, используется «порча» байтов, то есть некоторые байты валидных входных тестов меняются на случайные.

4.2. Итоги

По итогу обзора статей, выделен общий паттерн фаззера, который будет эффективен при тестировании шаблонизаторов:

- Grammar-based
- Coverage-guided
- Использует обратную связь от целевой программы для направления процесса мутации
- В качестве метрики для отбора семян для генерации следующего поколения тестов используется покрытие кода, а генерация тестов происходит на основе

вероятностной грамматики.

В качестве фаззера было решено использовать EvoGFuzz, так как он разрабатывался для тестирования любых программ и движков, которые принимают на вход сложные, структурированные данные. С помощью него планируются генерировать тесты с помощью вероятностной грамматики. Также полезным инструментом будет Grammarinator, который является своего рода state-of-the-art решением в тестировании компиляторов и движков. С помощью встроенной функции мутации планируется перемешивать тесты с целью получения интересных шаблонов и большего покрытия кода.

5. Экспериментальные исследования

В качестве тестируемого фреймворка был выбран Smarty, ввиду его популярности. Данный шаблонизатор не предоставляет никакой автоматической защиты от XSS, поэтому все отображаемые данные должны быть экранированы с помощью модификатора `escape`. Таким образом, цель исследования - проверить методы фазз-тестирования при поиске недостатков, связанных с экранированием данных.

Как было выяснено при обзоре статей, для генерации шаблонов необходимо построить грамматику Smarty. В ее поисках были рассмотрены различные интернет-ресурсы, но ничего найдено не было. Поэтому она была построена вручную, на основе некоторого представления грамматики, хранящейся в комментариях к коду самого шаблонизатора. По итогу грамматика была разработана в двух экземплярах - в формате `bnf` и `antlr4`. Первый вариант использовался для инструмента `tribble`, взятом из `EvoGFuzz`, который способен работать с вероятностным распределением. Вероятности для правил были составлены эмпирическим путем и преследовали цель увеличить то количество конструкций шаблонизатора, которые ответственны за экранирование данных. Второй предназначен для `Grammarinator`'а, с помощью него представляется возможным не только генерировать, но и мутировать тесты.

В процессе тестирования возникали разного рода ошибки: "PHP Warning: Division by zero", "PHP Warning: A non-numeric value encountere", "Smarty error: syntax error". Ошибки, возникающие при процессе фаззинга, рассматривались вручную.

Нерешенной проблемой в рамках данной работы стало отсутствие автоматического запуска тестируемой программы на входных данных, так как для проверки корректности экранирования необходимо было проверять, не запускался ли скрипт, при открытии интернет-страницы.

5.1. Покрытие кода

Для получения покрытия был использован инструмент `Xdebug` версии 2.9.2. При выполнении кода Smarty, генерируется файл, с информацией о выполненных строках кода. Для сбора информации о покрытии использовался инструмент `phpunit/php-code-coverage` версии 9.2. Вся информация была доступна через `html`-файл, где можно было просмотреть

сколько строк кода было выполнено, и каков процент покрытия для отдельных файлов. Поэтому различные техники сравнивались по тому, сколько строк покрыли сгенерированные ими тесты.

- Метод k-path

В ходе экспериментов были применены различные техники тестирования. Первой из них было k-path покрытие грамматики, взятое из статьи 13. Запуск выполнялся с помощью строки:

```
java -jar ./tribble/tribble.jar generate --suffix = .tpl --mode = k-path --out-dir = tpls --grammar-file = smarty7.bnf
```

Данная техника довольно эффективна при построении тестов, и с увеличением параметра k она генерирует больше файлов и покрывает больше кода. Но у нее есть недостаток - экспоненциальный взрыв входных данных при увеличении параметра, так что в данной работе рассматривались значения только 1, 2 и 3. Для исследования, было сгенерировано по 2 корпуса тестов для каждого параметра k, значение покрытия бралось среднее среди двух итераций. Результаты видны на графиках:

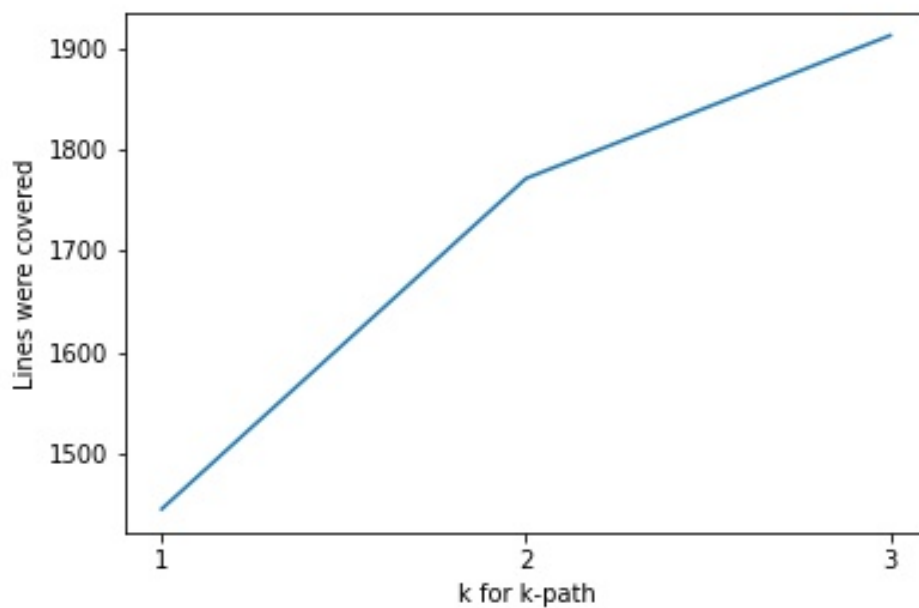


Рис. 6: Покрытие строк кода

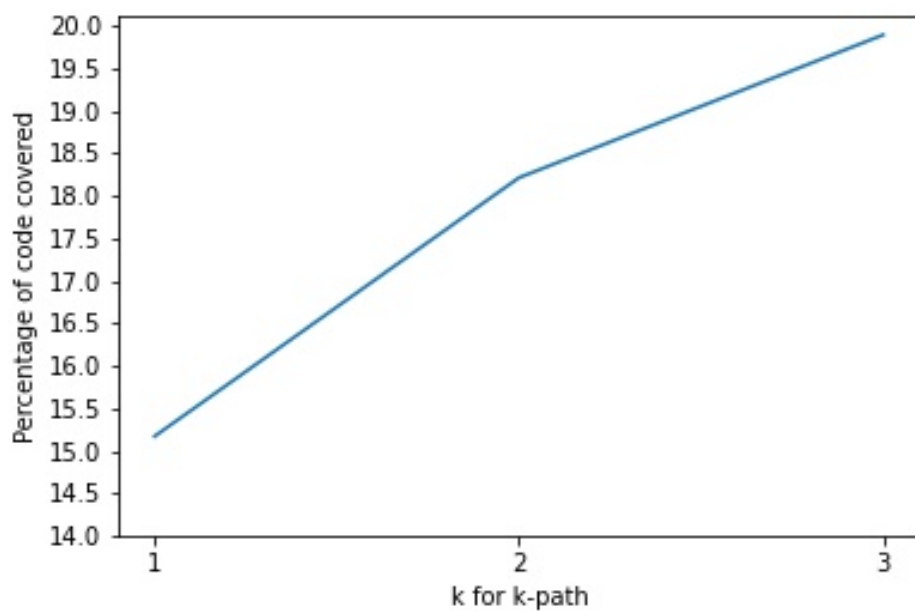


Рис. 7: Покрытие в процентах

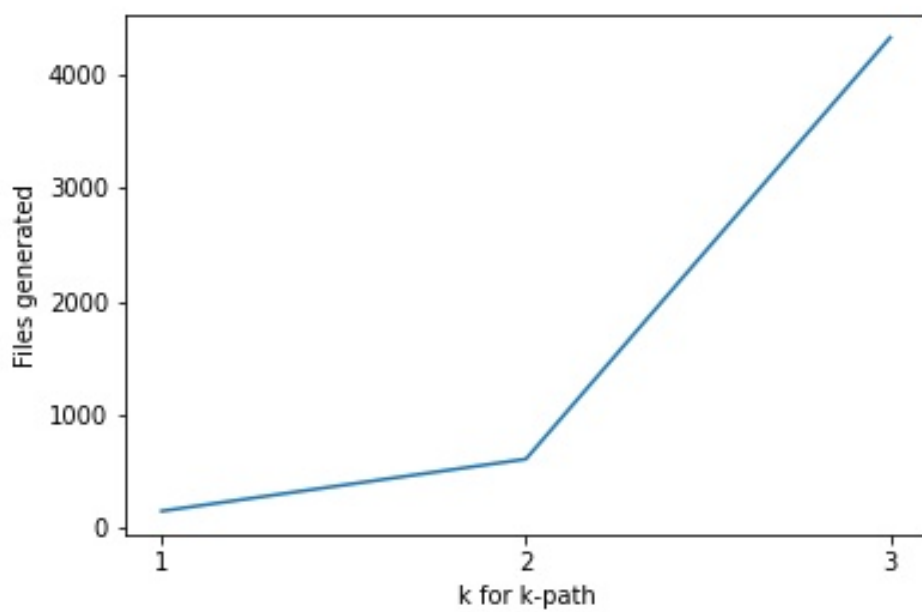


Рис. 8: Количество сгенерированных файлов

- Вероятностная грамматика

Другой подход, взятый из статьи 14, основан на вероятностной грамматике. Его

результат в ходе тестирования показал большее покрытие кода. Для его измерения было сгенерировано 5 поколений экземпляров по 50 тестов. На рисунке 9 продемонстрировано, какое количество строк было покрыто данной техникой. Запуск генерации происходил со следующими настройками:

`java -Xss1g -Xms256m -Xmx4g -jar./tribble/tribble.jargenerate --suffix = .tpl --mode = d - c - probabilistic - n --out - dir = tpls --grammar - file = smarty7.bnf`, где *d* - максимальная глубина дерева, построенная с помощью грамматики, *c* - глубина дерева, после которого, ищется максимально короткий путь, *n* - количество файлов для генерации.

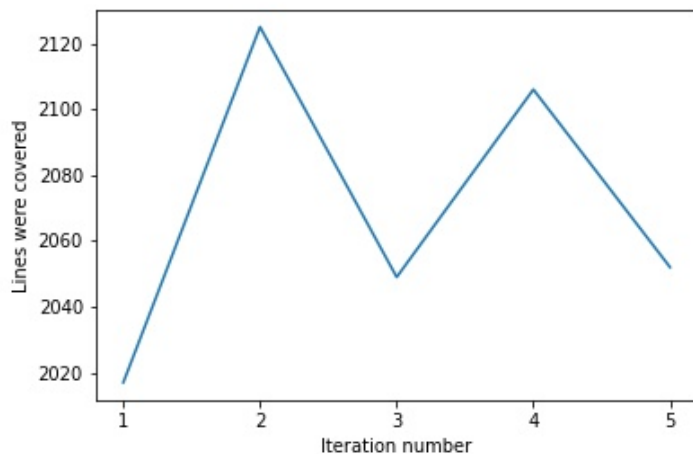


Рис. 9: Покрытие кода вероятностной грамматикой

Оба перечисленных подхода были реализованы с помощью инструмента *tribble*. Для сравнения была взята программа *Grammarinator*, популярная в использовании при фаззинг-тестировании. Но каких-то качественных улучшений результатов добиться не удалось. Как видно из графиков, среднее количество покрытых строк - от 2000 до 2200.

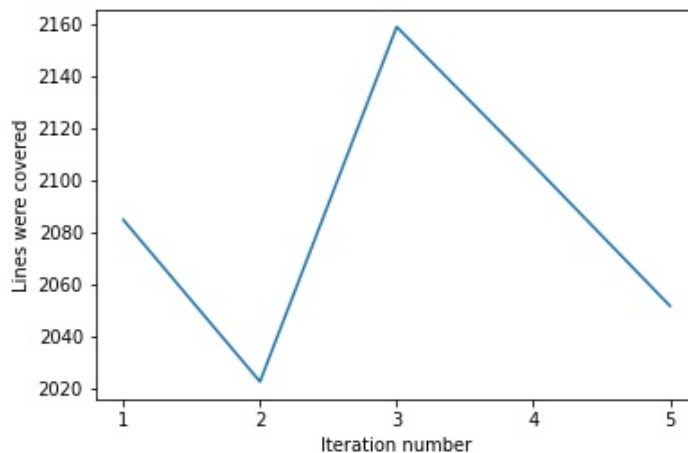


Рис. 10: Покрывтие кода Grammarinator'ом

5.2. Найденные недостатки

- Выполнение кода в явно объявленной строковой переменной

При объявлении строковой переменной, содержащей в себе конструкции шаблонизатора, код выполняется при обработке.

```
{ $tmp = "{ 5 is div by $user }" }
{ $tmp|date_format|escape:'html' }
```

На данном входе Smarty выдает ошибку *"PHP Fatalerror : Uncaught DivisionByZeroError : Modulo by zero"*, которая возникает при запуске скомпилированного файла. Это значит, что данный пример проходит все стадии анализа, но строка "{ 5 is div by \$user }" воспринимается как набор инструкций для шаблонизатора.

- Отсутствие экранирования при тернарном операторе.

При обработке следующего шаблона, экранирование переменной \$owner не произойдет:

```
{ (8) ? $owner : (6 * 9)|nl2br|escape:'html' }
```

Если переменная содержит строку, типа `'<script>alert("Ben")</script>'`, то скрипт будет выполнен.

- Интерпретация конструкции шаблонизаторов как текст

При запуске Smarty на следующем шаблоне, конструкция не будет выполнена:

```
{ $tmp|escape}
```

Если посмотреть на результат обработки шаблона, то в качестве выхода получится просто html-страница с текстом, идентичным содержанию шаблона. Это происходит из-за пробела после открытия фигурной скобки.

5.3. Интерпретация результатов

Недостаток, связанный с отсутствием экранирования в тернарном операторе, легко устраняется с помощью взятия в скобки условия, например:

```
{{(8)?$owner : (6 * 9))|nl2br|escape : 'html'}}
```

Тогда модификаторы будут применяться уже к результату выполнения оператора.

Что касается неинтерпретирования конструкции шаблонизатора - пользователи фреймворка Smarty должны быть просто внимательны и не использовать никаких отступов от фигурной скобки, тогда проблема устраняется.

Недостаток, связанный с выполнением кода в строке с кавычками, видимо связан с тем, что при обрамлении строки в фигурные скобки, кавычки игнорируются, и содержимое интерпретируется как код. Данная проблема устраняется тем, что разработчики просто не будут использовать никаких конструкций шаблонизатора в переменных.

5.4. Итоги эксперимента

По итогам экспериментов, были получены следующие результаты:

- Метод k-path не очень хорошо подходит для тестирования шаблонизатора, так как при маленьких параметрах k сгенерированные шаблоны очень просты и покрывают малое количество кода. При больших же k количество файлов возрастает экспоненциально. При параметре, равном 5, их свыше 90000, а при больших параметрах на компьютере не хватало памяти для хранения. Это делает невозможным запуск Smarty на всех полученных экземплярах вручную, как было сделано в рамках данной работы. Необходима доработка в виде автоматического запуска шаблонизатора на всех тестах.

- Для борьбы с бесконечной рекурсией при генерации тестов следует использовать особый режим инструмента *tribble*, который при достижении заданной глубины рекурсии выбирает правила, ведущие к терминальным символам. Например: *java -jar./tribble/tribble.jargenerate - -suffix = .tpl - -mode = d - c - probabilistic - n*
- Вероятностная грамматика показала лучшие результаты, по сравнению с методом *k-path*. При регулировке вероятностей правил можно сгенерировать шаблоны с более интересными конструкциями. Также при уменьшении вероятности пустого выхода в рекурсивных правилах стало возможным генерировать экземпляры с большим набором правил грамматики. Однако существенным недостатком является то, что вероятности подкручивались вручную, поэтому остается открытой проблема их изменения на основе обратной информации от шаблонизатора.
- Использование безвероятностной грамматики с помощью инструмента *Grammarinator* привело к получению результатов не хуже, чем с использованием вероятностей. Данный результат можно объяснить несколькими причинами: неудачной расстановкой вероятностей, недостаточное количество сгенерированных тестов. Либо из-за того, что количество кода, которое способно покрыть фазз-тестирование, ограничено, ввиду существования различных функций шаблонизатора, обрабатывающих не сам шаблон, а иные файлы. Поэтому все пути исполнения, которые могли быть вызваны, были покрыты обоими техниками.

Также был установлен ряд недостатков примененного фазз-процесса:

- Построенная грамматика иногда выдавала неправильно составленные шаблоны. Это произошло, потому что она была построена вручную. Некоторые правила из некоего представления формальной грамматики в коде шаблонизатора были отброшены из-за невозможной их интерпретации. Таким образом предполагается, что некоторые пути исполнения не были покрыты.
- Общее покрытие кода в результате тестирования получилось довольно небольшим, не более 21%. Это произошло как в силу неполноты построенной грамматики, так и невозможностью фазз-тестирования вызвать строки кода, ответственные за работу с внешними по отношению к шаблону объектами. Например, тот код, который

ответственен за работу с конфигурационными или дочерними шаблонами, не был затронут.

- При создании шаблонов на основе первых экземплярах грамматики возникла проблема с генерацией переменных, их названия получались случайными. Поэтому при построении фаззинга нужно заранее продумать семантические связи, чтобы они не нарушались при фазз-процессе. Для этого можно настроить соответствующим образом грамматику, которая бы сохраняла их, либо после генерации шаблона обрабатывать такие случаи, например: все случайные переменные заменить либо на зафиксированные в коде веб-приложения, либо на случайные значения.
- Вероятность для правил грамматики была подобрана вручную, а не получена на основе большого корпуса примеров. Таким образом, использованные вероятности могли быть расставлены не самым подходящим способом. Также из-за этого нельзя было использовать метод инверсии вероятностей, описанных в статье 14.
- Не использование обратной связи от шаблонизатора при генерации тестового корпуса ввиду отсутствия соответствующего инструмента.
- Неавтоматизированность процесса фаззинга. Для запуска Smarty на шаблонах использовалось ручное тестирование, что затрачивало большое количество времени.
- Подвергнуть мутации шаблоны, сгенерированные с помощью tribble, не удалось. Парсер Grammarinator не смог построить AST-деревья из таких тестов. Причиной может служить неэквивалентность построенных грамматик. Для построения в формате antlr4, необходимо создать два файла - один с лексемами, другой с правилами для парсера. Лексемы для этого формата видимо были отличны от тех, что использовались в грамматике bnf, которые присутствуют непосредственно в правилах.

5.5. Вывод

Исходя из полученных результатов, можно сделать следующие выводы:

- Неудовлетворительно низкое покрытие кода. С помощью метода фазз-тестирования получилось проверить только ту часть кода шаблонизатора, которая ответственна

за парсинг и работу с конструкциями языка движка шаблонов. Другие пути исполнения оно покрыть неспособно, так как не затрагивает сам код веб-приложения или конфигурационные файлы. Для исправления данной проблемы необходимо изучить все возможности шаблонизатора и подключить их при фазз-тестировании, либо при каждой новой итерации фаззинга подключать их попеременно.

- Процесс фазз-тестирования нельзя полностью автоматизировать, так как приходится вручную изменять вероятности правил грамматики и запускать код шаблонизатора путем открытия страницы через браузер.
- При отсутствии в открытом доступе формальной грамматики шаблонизатора, ставится под сомнение возможность проверить поведение программы на всех возможных шаблонах.

В качестве следующих направлений работы предлагается:

- Автоматизировать анализ ошибок: синтаксические ошибки и различные ошибки выполнения математических операций игнорировать, а другие складывать в отдельную директорию с целью ручного анализа и генерации нового поколения тестов на их основе.
- Разработка метода мутации для шаблонов, в результате которого будут получены валидные с точки зрения синтаксиса тесты.
- Разработка инструмента, который будет на основе полученной информации от шаблонизатора отбирать наиболее интересные тесты с точки зрения заданной fitness-функции. Исследование и изучение такого рода функций при тестировании обработчика шаблонов.

6. Итоги

В рамках данной работы, были выполнены следующие задачи:

- Изучены статьи о фазз-тестировании
- Составлен общий паттерн фазз-инструмента для шаблонизатора
- Найдены подходящие инструменты для тестирования
- Составлена грамматика Smarty
- Применен метод фазз-тестирования на шаблонизаторе Smarty, использованы его различные приемы
- Найдены и проинтерпретированы недостатки шаблонизатора
- Сделаны выводы о применимости фазз-тестирования касательно шаблонизаторов
- Обозначены дальнейшие направления работы

Список литературы

- [1] <https://www.cscjournals.org/manuscript/Journals/IJCSS/Volume13/Issue3/IJCSS-1481.pdf>
- [2] <https://arxiv.org/pdf/2008.01150.pdf>
- [3] <https://i.blackhat.com/asia-19/Fri-March-29/bh-asia-Dominiak-Efficient-Approach-to-Fuzzing-Interpreters-wp.pdf>
- [4] <https://scihubtw.tw/10.1145/3278186.3278193>
- [5] <https://arxiv.org/pdf/1701.07232.pdf>
- [6] <https://www.smarty.net/>
- [7] <https://twig.symfony.com/>
- [8] <https://laravel.ru/docs/v5/blade>
- [9] <https://docs.phalcon.io/4.0/en/volt>
- [10] <https://freemarker.apache.org/docs/index.html>
- [11] <https://velocity.apache.org/>
- [12] <https://www.thymeleaf.org/>
- [13] https://havrikov.github.io/publications/ase2019_havrikov.pdf
- [14] <https://havrikov.github.io/publications/tse2020-preprint.pdf>